Documentação Técnica — Mappers

Padrão de Conversão DTO ↔ Entity no Barber System

Projeto:	Barber System
Assunto:	Mappers manuais (padrão estático) — toEntity, toSaidaDto, atualizarEntity
Camadas envolvidas:	Controller → Service → Mapper → Entity / DTO
Data:	Setembro/2026

1. O que é um Mapper e por que usamos?

Um **Mapper** é uma classe utilitária responsável por **converter dados entre camadas** da aplicação. No Barber System, os Mappers fazem a ponte entre os **DTOs** (objetos que trafegam na API / formulários) e as **Entities** (objetos que representam as tabelas do banco de dados).

Por que não devolver a Entity diretamente na API?

- A Entity pode conter campos sensíveis (senha, relacionamentos lazy, campos internos).
- A Entity está acoplada ao JPA/Hibernate; mudar o banco quebraria a API.
- O DTO de saída controla exatamente o que o cliente (frontend) recebe.
- O DTO de entrada valida apenas os campos que o usuário pode enviar.

Decisão do projeto: Mappers **manuais e estáticos** (sem MapStruct ou ModelMapper). Isso dá controle total sobre o que é copiado, evita dependências extras e deixa o código transparente para estudo.

2. Estrutura padrão dos Mappers no projeto

Todos os Mappers do Barber System seguem a mesma estrutura de métodos. Isso padroniza o código e facilita a manutenção:

Método	Direção	Quando é usado	Camada que chama
toEntity(...)	DTO → Entity	Criação de um novo registro	Service (no método de cadastro)
toSaidaDto(...)	Entity → DTO	Retorno de um único registro para a API	Service (após buscar/salvar)
toSaidaDtoList(...)	List<Entity> → List<DTO>	Listagens (findAll, filtros, etc.)	Service (métodos de listagem)
atualizarEntity(...)	DTO atualização → Entity	Atualização parcial (PATCH-like)	Service (método de update)

Classes Mapper existentes: **ClienteMapper**, **BarbeiroMapper**, **ServicoMapper**, **AgendamentoMapper** e (esqueleto) **HararioDeTrabalhoMapper**.

3. Fluxo por camada — quem chama o quê

3.1. Cadastro (criação)

Controller recebe o DTO de entrada → Service valida regras de negócio → Service chama `Mapper.toEntity(dto)` → Service completa campos (senha criptografada, status, nível de acesso) → Repository salva a Entity → Service chama `Mapper.toSaidaDto(entity)` e devolve para o Controller.

```
// Exemplo no ClienteService.cadastroCliente(...)
ClienteEntity entity = ClienteMapper.toEntity(dto);          // 1. DTO → Entity
entity.setSenha(passwordEncoder.encode(dto.senha()));        // 2. Service trata senha
entity.setNivelAcesso(NivelAcesso.CLIENTE);                  // 3. Service define defaults
entity.setStatus(Status.ATIVO);
repository.save(entity);                                     // 4. Persiste
return ClienteMapper.toSaidaDto(entity);                     // 5. Entity → DTO de saída
```

3.2. Listagem / busca

Service busca no Repository (retorna Entity ou List<Entity>) → chama `toSaidaDto` ou `toSaidaDtoList` → Controller devolve o DTO (nunca a Entity).

3.3. Atualização parcial

Service busca a Entity pelo ID → valida permissões → chama `Mapper.atualizarEntity(dto, entity)` (só sobrescreve campos não-nulos) → Repository salva → retorna `toSaidaDto(entity)`.

3.4. Alteração de senha

> **Não passa pelo Mapper.** Toda a lógica fica no Service: validar senha atual com `passwordEncoder.matches`, validar confirmação, gerar novo hash e salvar. Isso é regra de negócio/segurança, não mapeamento de campos.

4. Explicação detalhada de cada método

4.1. toEntity — DTO de entrada → Entity

Objetivo: criar uma nova Entity a partir dos dados enviados pelo cliente na criação.

Por que existe:

- Isola a conversão de campos; o Service não fica cheio de `setNome`, `setEmail`, etc.
- Centraliza a lógica de "quais campos do DTO vão para a Entity na criação".
- Protege contra NullPointerException com `if (dto == null) return null`.

O que NÃO deve ser feito no toEntity:

- **Não setar senha.** A senha deve ser criptografada no Service com PasswordEncoder.
- Não setar ID (é gerado pelo banco).
- Não setar status / nível de acesso (regras de negócio do Service).
- Não buscar outras entidades (isso é responsabilidade do Service).

```
// ClienteMapper.toEntity — exemplo real do projeto
public static ClienteEntity toEntity(ClienteDto dto) {
    if (dto == null) return null;
    ClienteEntity entity = new ClienteEntity();
    entity.setNome(dto.nome());
    entity.setDataNascimento(dto.dataNascimento());
    entity.setTelefone(dto.telefone());
    entity.setEmail(dto.email());
    // entity.setSenha(...) → propositalmente NÃO feito aqui
    return entity;
}
```

4.2. toSaidaDto — Entity → DTO de saída

Objetivo: expor apenas os dados seguros e úteis para o frontend / consumidor da API.

Por que existe:

- **Segurança:** a senha (hash) nunca sai da Entity para o DTO de saída.
- **Contrato da API:** o frontend sabe exatamente quais campos receberá.
- **ID no DTO de saída:** necessário para montar links (editar, desativar, etc.) e identificar o registro.
- Evita expor relacionamentos lazy ou estruturas internas do JPA.

```
// ClienteMapper.toSaidaDto — exemplo real
public static ClienteSaidaDto toSaidaDto(ClienteEntity entity) {
    if (entity == null) return null;
    return new ClienteSaidaDto(
        entity.getId(),
        entity.getNome(),
        entity.getDataNascimento(),
        entity.getTelefone(),
        entity.getEmail(),
        entity.getStatus()
        // senha NÃO é incluída de propósito
    );
}
```

4.3. toSaidaDtoList — Lista de Entities → Lista de DTOs

Objetivo: converter coleções de forma consistente, reaproveitando o `toSaidaDto` individual.

Por que existe:

- Evita duplicar o loop de conversão em todo método de listagem do Service.
- Trata lista nula retornando `List.of()` (lista vazia imutável), evitando NPE.
- Usa Stream + method reference (`Mapper::toSaidaDto`) — código limpo e idiomático.

```
public static List<ClienteSaidaDto> toSaidaDtoList(List<ClienteEntity> entities) {
    if (entities == null) {
        return List.of();
    }
    return entities.stream()
            .map(ClienteMapper::toSaidaDto)
            .toList();
}
```

4.4. atualizarEntity — DTO de atualização → Entity existente

Objetivo: aplicar apenas os campos enviados no DTO de atualização sobre a Entity já carregada do banco (atualização parcial / estilo PATCH).

Por que existe:

- O usuário pode querer alterar só o telefone, sem reenviar nome, e-mail, etc.
- Cada campo é atualizado só se `dto.campo() != null` — campos omitidos não são zerados.
- Mantém a Entity gerenciada pelo JPA; o Service só chama `repository.save(entity)` depois.
- Separação clara: Mapper só copia campos; Service valida permissões e unicidade de e-mail/telefone.

```
// ClienteMapper.atualizarEntity — exemplo real
public static void atualizarEntity(ClienteAtualizaDto dto, ClienteEntity entity) {
    if (dto == null || entity == null) return;

    if (dto.nome() != null)          entity.setNome(dto.nome());
    if (dto.dataNascimento() != null) entity.setDataNascimento(dto.dataNascimento());
    if (dto.telefone() != null)      entity.setTelefone(dto.telefone());
    if (dto.email() != null)         entity.setEmail(dto.email());
    // senha NÃO é tratada aqui
}
```

Observação de nomenclatura: em ClienteMapper e BarbeiroMapper o método se chama `atualizarEntity`; em ServicoMapper se chama `atualizaEntity`. O ideal é padronizar o nome em todos os Mappers.

5. Por que alteração de senha NÃO fica no Mapper?

Nos Mappers de Cliente e Barbeiro há um comentário explícito documentando a ausência de método para `AlteraSenhaDto`. Isso é intencional e importante em produção.

Troca de senha envolve:

1. Validar a senha atual (comparar hash com `passwordEncoder.matches`) → regra de segurança.
2. Gerar hash da nova senha → depende do PasswordEncoder (injetado no Service).
3. Possivelmente invalidar sessões/tokens antigos.
4. Verificar se a nova senha é diferente da atual e se confere com a confirmação.

Nada disso é "mapeamento de campos". É **lógica de negócio e segurança**. Por isso a responsabilidade fica inteiramente no **Service** (ClienteService / BarbeiroService), e não no Mapper. Documentar essa ausência evita que alguém "complete" o Mapper achando que faltou implementar o método.

6. Caso especial: AgendamentoMapper

O Agendamento é diferente porque a Entity possui **relacionamentos** (Cliente, Barbeiro, Serviço). O DTO de entrada traz apenas IDs; o Service busca as Entities e passa para o Mapper.

toEntity do Agendamento:

Recebe o DTO + as três Entities já carregadas. O Mapper monta a Entity, calcula `dataHoraFim` com base na duração do serviço e define o status inicial `AGENDADO`.

```
// AgendamentoMapper.toEntity — assinatura especial
public static AgendamentoEntity toEntity(
        AgendamentoDto dto,
        ClienteEntity cliente,      // já buscado no Service
        BarbeiroEntity barbeiro,    // já buscado no Service
        ServicoEntity servico) {    // já buscado no Service

    AgendamentoEntity entity = new AgendamentoEntity();
    entity.setCliente(cliente);
    entity.setBarbeiro(barbeiro);
    entity.setServico(servico);
    entity.setDataHoraInicio(dto.dataHoraInicio());

    // Calcula fim com base na duração do serviço
    if (servico != null && servico.getDuracaoEmMinutos() != null) {
        entity.setDataHoraFim(
            dto.dataHoraInicio().plusMinutes(servico.getDuracaoEmMinutos()));
    }
    entity.setObservacao(dto.observacao());
    entity.setStatusAgendamento(StatusAgendamento.AGENDADO);
    return entity;
}
```

toSaidaDto do Agendamento:

"Achata" os relacionamentos: em vez de devolver objetos aninhados, extrai `clienteId`, `clienteNome`, `barbeiroId`, `barbeiroNome`, dados do serviço, datas e status. Isso evita problemas de serialização lazy e entrega um JSON plano e legível.

Por que o Service busca as Entities e não o Mapper? Porque buscar no banco é responsabilidade da camada de serviço/persistência. O Mapper deve apenas copiar e montar objetos em memória, sem acessar Repository ou banco.

7. Resumo por Mapper do projeto

Mapper	toEntity	toSaidaDto	toSaidaDtoList	atualizarEntity	Obs.
ClienteMapper	Sim	Sim	Sim	Sim	Senha só no Service
BarbeiroMapper	Sim	Sim	Sim	Sim	Senha só no Service
ServicoMapper	Sim	Sim	Sim	Sim (atualizaEntity)	Sem senha
AgendamentoMapper	Sim (com Entities)	Sim (achatado)	Sim	Não	Relacionamentos no Service
HararioDeTrabalhoMapper	Esqueleto	Esqueleto	Esqueleto	Esqueleto	Ainda não implementado

8. Boas práticas aplicadas (e recomendações)

Prática	Por que
Métodos estáticos + construtor privado/vazio	Mapper é utilitário; não precisa de instância nem de estado
Proteção contra null no início de cada método	Evita NullPointerException em cadeias de chamada
Senha fora do Mapper	Criptografia e validação são regra de negócio, não mapeamento
atualizarEntity só com campos não-nulos	Permite atualização parcial sem apagar dados existentes
toSaidaDtoList reutiliza toSaidaDto	DRY — uma única fonte de verdade para a conversão
Agendamento: Service busca, Mapper monta	Mapper não acessa banco; responsabilidades bem separadas
Documentar o que NÃO existe no Mapper	Evita "completar" métodos que não devem existir (ex.: alterarSenha)

Recomendações de melhoria:

- Padronizar o nome `atualizarEntity` em todos os Mappers (hoje Servico usa `atualizaEntity`).
- Implementar de fato o `HararioDeTrabalhoMapper` seguindo o mesmo padrão.
- Manter o construtor privado (como no BarbeiroMapper) em todos, para reforçar que a classe não deve ser instanciada.

9. Diagrama mental do fluxo com Mapper

```
CADASTRO
  Frontend → Controller (DTO entrada)
           → Service (valida regras)
           → Mapper.toEntity(dto)          ← mapeamento
           → Service (encode senha, defaults)
           → Repository.save(entity)
           → Mapper.toSaidaDto(entity)     ← mapeamento
           → Controller → Frontend (DTO saída)

LISTAGEM
  Frontend → Controller
           → Service → Repository.find...
           → Mapper.toSaidaDtoList(...)    ← mapeamento
           → Controller → Frontend

ATUALIZAÇÃO
  Frontend → Controller (DTO atualização)
           → Service (busca entity, valida)
           → Mapper.atualizarEntity(dto, entity)  ← mapeamento
           → Repository.save(entity)
           → Mapper.toSaidaDto(entity)     ← mapeamento
           → Controller → Frontend

ALTERAR SENHA
  Frontend → Controller (AlteraSenhaDto)
           → Service (valida senha atual, encode, salva)
           → (Mapper NÃO participa)
```

10. Conclusão

Os Mappers do Barber System são a camada de **tradução** entre o mundo externo (DTOs da API) e o mundo interno (Entities do banco). Cada método tem um papel bem definido:

- **toEntity** → cria Entity na criação (sem senha, sem defaults de negócio).
- **toSaidaDto / toSaidaDtoList** → expõe dados seguros e estáveis para a API.
- **atualizarEntity** → aplica atualização parcial sobre a Entity já existente.
- **Senha e regras de negócio** → ficam no Service, nunca no Mapper.

Seguir esse padrão em novos Mappers (e no HararioDeTrabalhoMapper) mantém o código previsível, seguro e fácil de evoluir.

Documento gerado para o projeto **Barber System**
Versão 1.0 — Setembro/2026 — Baseado nos Mappers e no estudo em Documentação/Estudo-Fixação/Mappers.txt